SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I K Sathwik Kumar (192425051)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **K Sathwik Kumar**

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	

10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	**Automobile Industry**	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.

7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

1. Automobile Fault Diagnosis and Advisory Expert System Using Prolog

Abstract

The Automobile Fault Diagnosis and Advisory Expert System is a rule-based Artificial Intelligence system developed using SWI-Prolog. The system assists automobile service centers in identifying probable vehicle faults from symptoms such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. The system represents automobile knowledge using facts and production rules. Forward chaining is used for data-driven reasoning, while backward chaining is used for goal-driven reasoning. The system also demonstrates unification, backtracking, logical inference, diagnostic recommendations, and explanation of conclusions.

Introduction

Automobile service centers receive vehicles with different complaints and symptoms. Identifying the probable cause of a problem generally requires diagnostic knowledge and experience. An expert system can represent this knowledge using production rules and apply logical inference to identify possible faults. The proposed system accepts observed vehicle symptoms and provides a probable fault along with a recommended diagnostic action. The selected problem is based on the automobile industry scenario given in the assessment, which requires a Prolog expert system for vehicle fault diagnosis.

2. Problem Statement and Objectives

Problem Statement

A vehicle service center receives complaints such as:

- Engine overheating
- Difficulty starting
- Abnormal engine noise

Objectives

1. Develop an automobile fault diagnosis expert system using Prolog.
2. Represent automobile symptoms and faults as facts and rules.
3. Identify probable faults from observed symptoms.

4. Implement forward chaining.
5. Implement backward chaining.
6. Demonstrate unification and backtracking.
7. Recommend suitable diagnostic actions.

The assessment requires the system to demonstrate knowledge representation, production rules, forward and backward chaining, Prolog implementation, and industry-based testing.

3. Domain Analysis and Knowledge Acquisition

Domain Analysis

The selected domain is **Automobile Fault Diagnosis**.

Entity	Description
Vehicle	Vehicle being diagnosed
Symptom	Observed vehicle problem
Fault	Probable cause
Diagnostic Action	Recommended inspection
Technician	Person performing diagnosis

Main Symptoms

- Slow cranking
- Starting problem
- Dim headlights
- High engine temperature
- Low coolant
- Battery warning light
- Abnormal engine noise
- Knocking sound

Possible Faults

- Weak battery
- Alternator fault
- Cooling-system fault
- Engine misfire
- Fuel-system fault
- Brake-system fault
- Engine mechanical fault
- Wheel alignment fault

```
46 ?- list_vehicles.

==========================================
          VEHICLES IN KNOWLEDGE BASE
==========================================
- car1
- car2
- car3
- car4
- car5
- car6
- car7
- car8
==========================================
true.
```

Knowledge Acquisition

- The knowledge base represents relationships between automobile symptoms, probable faults, and diagnostic actions.

For example:

- ✓ **IF** the vehicle has slow cranking, dim headlights, and a starting problem **THEN** the probable fault is a weak battery.
- ✓ Another example:

- ✓ **IF** engine temperature is high and coolant is low **THEN** the probable fault is a cooling-system fault.
- ✓ The assessment expects domain knowledge to be converted into suitable facts, predicates, relationships, and production rules.

4. Knowledge Representation and Production Rules

Knowledge Representation

Vehicle symptoms are represented using Prolog facts.

Example:

```
symptom(car1, slow_cranking).
symptom(car1, dim_headlights).
symptom(car1, starting_problem).
```

A production rule is represented as:

```
weak_battery(Car) :-
   symptom(Car, slow_cranking),
   symptom(Car, dim_headlights),
   symptom(Car, starting_problem).
```

This represents:

IF slow cranking AND dim headlights AND starting problem
THEN weak battery.

Production Rules

Rule 1 – Weak Battery

```
weak_battery(Car) :-
   symptom(Car, slow_cranking),
   symptom(Car, dim_headlights),
   symptom(Car, starting_problem).
```

Rule 2 – Alternator Fault

```
alternator_fault(Car) :-
   symptom(Car, battery_warning_light),
   symptom(Car, dim_headlights).
```

Rule 3 – Cooling-System Fault

```
cooling_fault(Car) :-
   symptom(Car, high_engine_temperature),
   symptom(Car, low_coolant).
```

Rule 4 – Engine Misfire

```
engine_misfire(Car) :-
   symptom(Car, rough_idle),
   symptom(Car, abnormal_vibration),
   symptom(Car, reduced_engine_power).
```

Rule 5 – Fuel-System Fault

```
fuel_system_fault(Car) :-
    symptom(Car, low_mileage),
    symptom(Car, poor_acceleration).
```

Rule 6 – Brake-System Fault

```
brake_fault(Car) :-
    symptom(Car, brake_noise),
    symptom(Car, reduced_braking).
```

Rule 7 – Engine Mechanical Fault

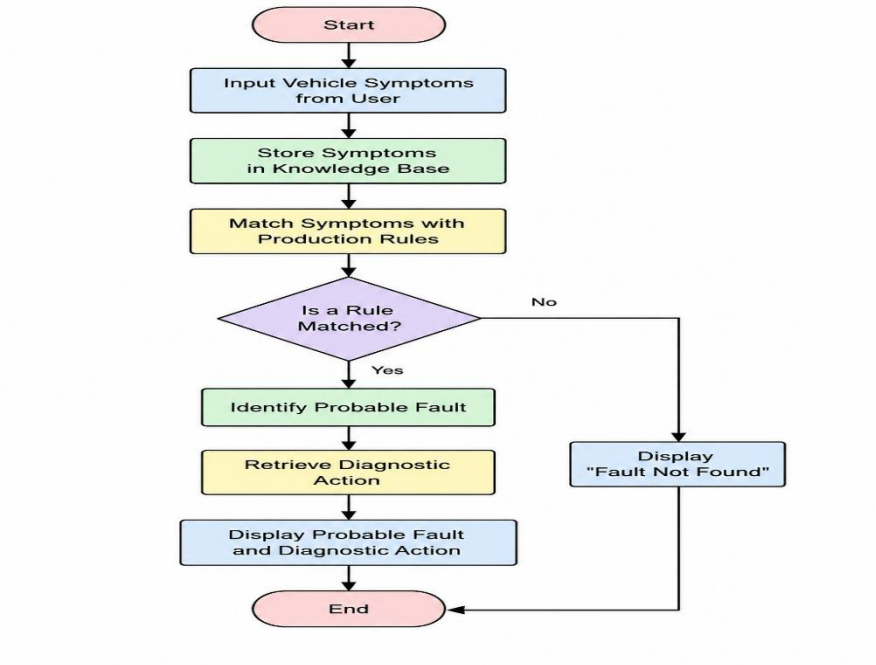
```
engine_mechanical_fault(Car) :-
    symptom(Car, abnormal_engine_noise),
    symptom(Car, knocking_sound).
```

Rule 8 – Wheel Alignment Fault

```
alignment_fault(Car) :-
    symptom(Car, vehicle_pulls_side),
    symptom(Car, steering_vibration).
```

The production-rule knowledge base is designed to connect observed symptoms with probable automobile faults. The assessment rubric emphasizes a comprehensive, consistent, and logically structured knowledge base.

5. Expert System Architecture



The architecture contains the user input, knowledge base, inference engine, diagnosis, and explanation/output components required for the expert-system design.

6. Inference Mechanism and Algorithms

Forward Chaining

Forward chaining is a **data-driven reasoning method**.

The system begins with known vehicle symptoms and checks the production rules.

Vehicle Symptoms
↓
Check Production Rules
↓
Matching Rule
↓
Derive Probable Fault
↓
Diagnostic Action

Example:

Starting Problem
↓
Weak Battery Rule
↓
Weak Battery
↓
Check Battery Voltage

In Prolog:

forward_chaining(car1).

Forward Chaining Output

```
48 ?- forward_chaining(car1).

==========================================
              FORWARD CHAINING
==========================================
Vehicle: car1

Known Symptoms:
- slow_cranking
- dim_headlights
- starting_problem

Reasoning:
Rule Matched: Weak Battery
Conclusion: Probable Weak Battery
Action: Check battery voltage and terminals
==========================================
```

Backward Chaining

Backward chaining is a **goal-driven reasoning method**.

The system begins with a suspected fault and checks whether the necessary symptoms are present.

Goal: Weak Battery
↓
Find Weak Battery Rule
↓
Check Slow Cranking
↓
Check Dim Headlights
↓
Check Starting Problem
↓
Conditions Satisfied
↓
Weak Battery Confirmed

```
49 ?- backward_chaining(car1).

==========================================
            BACKWARD CHAINING
==========================================
Vehicle: car1

Checking possible faults...
Goal: Weak Battery -> TRUE
Goal: Alternator Fault -> FALSE
Goal: Cooling System Fault -> FALSE
Goal: Engine Misfire -> FALSE
Goal: Fuel System Fault -> FALSE
Goal: Brake System Fault -> FALSE
Goal: Engine Mechanical Fault -> FALSE
Goal: Wheel Alignment Fault -> FALSE
==========================================
true.
```

7. Prolog Implementation

The expert system is implemented in **SWI-Prolog**.

The implementation consists of:

- Vehicle symptom facts
- Production rules
- Diagnostic actions
- Forward-chaining mechanism

Prolog Source Code

The complete implementation is contained in:

implementation2.pl

The program uses Prolog predicates and variables to match vehicle symptoms with production rules and derive probable faults.

Figure: Prolog Source Code

Insert the screenshot of the automobile_expert.pl implementation here.

The assessment gives significant weight to the Prolog implementation and expects facts, rules, predicates, variables, unification, backtracking, and inference to be demonstrated.

8. Unification and Backtracking

Unification

Unification allows a Prolog variable to match a value satisfying a rule.

Query:

?- weak_battery(Car).

Output:

Car = car1.

Here, Car is a variable that becomes unified with car1 because car1 satisfies the weak-battery rule.

Backtracking

Backtracking allows Prolog to search for additional solutions.

Query:

?- weak_battery(Car).

If more than one vehicle satisfies the rule, pressing:

;

asks Prolog to search for another solution.

Example:

Car = car1 ;

Car = car9 ;

false.

Thus, unification performs variable matching, while backtracking allows Prolog to explore alternative solutions.

The assessment requires unification and backtracking to be demonstrated as part of the inference process.

9. Procedural and Non-Procedural Paradigms

Procedural Approach	Prolog / Declarative Approach
Describes how to solve	Describes what is true
Uses explicit steps	Uses facts and rules
Programmer controls execution	Prolog performs inference
Procedures are central	Logical relationships are central

Procedural Example

1. Read vehicle symptoms.
2. Check starting condition.
3. Check battery condition.
4. Determine fault.
5. Display diagnostic action.

Non-Procedural Example

```
weak_battery(Car) :-
    symptom(Car, slow_cranking),
    symptom(Car, dim_headlights),
    symptom(Car, starting_problem).
```

The Prolog rule describes the relationship between symptoms and the probable fault without explicitly specifying a complete procedural sequence.

The assessment requires this distinction to be explained and demonstrated in the developed expert system.

10. Test Cases and Results

The system is tested using multiple automobile scenarios. The assessment requires a minimum of **five industry-based test cases** with queries, expected outputs, actual outputs, and inference traces.

Test Case	Symptoms	Expected Fault
TC01	Slow cranking, dim headlights, starting problem	Weak Battery
TC02	Warning light, dim headlights	Alternator Fault
TC03	High temperature, low coolant	Cooling-System Fault
TC04	Rough idle, vibration, reduced power	Engine Misfire
TC05	Low mileage, poor acceleration	Fuel-System Fault

TC01 – Weak Battery

Query:

```
?- diagnose(car1).
```

Expected Result:

```
Probable Fault    : weak_battery
Diagnostic Action : Check battery voltage and battery terminals
```

TC02 – Alternator Fault

```
?- diagnose(car2).
```

Expected Result:

```
Probable Fault    : alternator_fault
Diagnostic Action : Check alternator output and charging voltage
```

TC03 – Cooling-System Fault

```
?- diagnose(car3).
```

Expected Result:

```
Probable Fault    : cooling_fault
```

Diagnostic Action : Check coolant level, radiator and cooling system

TC04 – Engine Misfire

?- diagnose(car4).

Expected Result:

Probable Fault : engine_misfire

Diagnostic Action : Inspect spark plugs, ignition system and engine

TC05 – Fuel-System Fault

?- diagnose(car5).

Expected Result:

Probable Fault : fuel_system_fault

Diagnostic Action : Inspect fuel filter, fuel pump and injectors

11. Testing and Results Analysis

- The test cases demonstrate that the system can identify different probable faults from different combinations of symptoms.
- For car1, the presence of slow cranking, dim headlights, and a starting problem satisfies the weak-battery rule.
- For car2, the battery warning light and dim headlights satisfy the alternator-fault rule.
- For car3, high engine temperature combined with low coolant produces a cooling-system diagnosis.
- For car4, rough idle, abnormal vibration, and reduced engine power result in an engine-misfire diagnosis.
- For car5, low mileage and poor acceleration result in a fuel-system diagnosis.

12. Diagnostic Actions

Fault	Recommended Action
Weak Battery	Check battery voltage
Alternator Fault	Check charging voltage
Cooling Fault	Check coolant and radiator
Engine Misfire	Inspect spark plugs and ignition
Fuel-System Fault	Inspect fuel system
Brake Fault	Inspect brake components
Mechanical Fault	Inspect engine components
Alignment Fault	Check tires and alignment

13. Explanation Facility

The explanation facility provides the reason behind a diagnosis.

For example:

Observed Symptoms:

- slow_cranking
- dim_headlights
- starting_problem

Reason:
These symptoms satisfy the weak-battery production rule.
Probable Fault:

- Weak Battery
- Recommended Action:
- Check battery voltage and battery terminals.
- The explanation facility makes the system's reasoning easier for a service technician to understand.

14. Limitations

1. The system depends on predefined production rules.
2. It does not automatically learn new diagnostic rules.
3. Different faults may sometimes produce similar symptoms.
4. The system provides probable faults rather than guaranteed mechanical diagnoses.
5. Physical inspection is still required for final confirmation.
6. The accuracy depends on the completeness of the knowledge base.

The assessment also expects limitations and realistic future enhancements to be discussed.
15. Future Enhancements

1. Add more automobile faults and symptoms.
2. Include vehicle make and model.
3. Add numerical sensor readings.
4. Include vehicle maintenance history.
5. Add fault severity levels.
6. Add confidence scores.
7. Develop a graphical user interface.
8. Connect the system with a vehicle diagnostic database.
9. Add more complex production rules.
10. Integrate machine-learning techniques for improved diagnosis.

16. Conclusion

- The **Automobile Fault Diagnosis and Advisory Expert System** demonstrates the application of Artificial Intelligence and Prolog to an industry-based automobile diagnostic problem.
- The system represents vehicle symptoms as facts and uses production rules to identify probable faults.
- Forward chaining provides data-driven reasoning, while backward chaining provides goal-driven reasoning.
- The system also demonstrates important Prolog concepts including unification, backtracking, logical inference, and automated conclusion generation.
- Testing with multiple automobile scenarios demonstrates that the system can identify different probable faults and provide suitable diagnostic actions.

17. References

1. SWI-Prolog documentation.
2. Bratko, I., *Prolog Programming for Artificial Intelligence*.
3. Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*.
4. Course materials for Artificial Intelligence and Expert Systems.
5. Automobile fault-diagnosis domain references.

The new assessment requires references and supporting sources as part of the report and documentation requirements.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.

5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference	7–8% – Provides relevant test cases with mostly	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.

Total	100%				